

# Tuyển Tập Bài Tập: Đánh Giá Độ Phức Tạp Thuật Toán & Tối Ưu Hóa Hiệu Năng

(Curated Competitive Programming Problems on Complexity Analysis, Amortized Bounds & Pruning)

**Tác giả:** Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

**Soạn thảo:** Được hỗ trợ và soạn thảo bởi Agentic AI tool

**Bản quyền** © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

## Bảng Tổng Hợp 16 Bài Tập Tuyển Chọn

| STT | Mã Bài    | Tên Bài Toán                            | Nguồn      | Độ Khó | Chủ Đề Trọng Tâm                                                    |
|-----|-----------|-----------------------------------------|------------|--------|---------------------------------------------------------------------|
| 1   | CSES 1621 | <a href="#">Distinct Numbers</a>        | CSES       | ☆☆☆    | So sánh $O(N \log N)$ Sorting vs $O(N)$ Hash Map                    |
| 2   | CSES 1095 | <a href="#">Exponentiation</a>          | CSES       | ☆☆☆    | Phân tích $O(\log B)$ lũy thừa nhị phân                             |
| 3   | CSES 1083 | <a href="#">Missing Number</a>          | CSES       | ☆☆☆    | Tối ưu không gian: $O(N)$ thời gian, $O(1)$ RAM bằng phép XOR       |
| 4   | CSES 1068 | <a href="#">Weird Algorithm</a>         | CSES       | ☆☆☆    | Dãy Collatz & Độ phức tạp tiệm cận chưa chứng minh                  |
| 5   | CSES 1624 | <a href="#">Chessboard and Queens</a>   | CSES       | ☆☆☆    | Phân tích nhánh cận $8! = 40320$ trạng thái của bài toán 8 quân hậu |
| 6   | CSES 1640 | <a href="#">Sum of Two Values</a>       | CSES       | ☆☆☆    | Hai con trỏ $O(N \log N)$ vs Bảng băm an toàn $O(N)$ amortized      |
| 7   | CSES 1644 | <a href="#">Maximum Subarray Sum II</a> | CSES       | ☆☆☆☆   | Phân tích khấu hao Monotonic Deque $O(N)$ trên cửa sổ trượt         |
| 8   | CF 1213G  | <a href="#">Path Queries</a>            | Codeforces | ☆☆☆☆   | Phân tích độ phức tạp DSU Union by Size $O((N + Q)\alpha(N))$       |
| 9   | CSES 1081 | <a href="#">Maximum GCD</a>             | CSES       | ☆☆☆☆   | Phân tích Chuỗi điều hòa (Harmonic Series) $O(V \ln V)$             |
| 10  | CF 1338B  | <a href="#">Edge Weight Assignment</a>  | Codeforces | ☆☆☆☆   | Phân tích tính chặn lẻ đường đi cây trong $O(V + E)$ 1 lượt DFS     |

| STT | Mã Bài           | Tên Bài Toán                                      | Nguồn      | Độ Khó | Chủ Đề Trọng Tâm                                                   |
|-----|------------------|---------------------------------------------------|------------|--------|--------------------------------------------------------------------|
| 11  | <b>CSES 1628</b> | <a href="#">Meet in the Middle</a>                | CSES       | ★★★★★  | Phân rã $N = 40$ : Giảm từ $O(2^{40})$ xuống $O(2^{20} \cdot 20)$  |
| 12  | <b>CF 1405E</b>  | <a href="#">Fixed Point Removal</a>               | Codeforces | ★★★★★  | Tối ưu hóa từ $O(N^2)$ xuống $O((N + Q) \log N)$ bằng Walk on Tree |
| 13  | <b>CF 1144F</b>  | <a href="#">Graph Without Long Directed Paths</a> | Codeforces | ★★★★★  | Kiểm tra đồ thị hai phía trong $O(V + E)$ chuyển hướng cạnh        |
| 14  | <b>CSES 1688</b> | <a href="#">Company Queries II</a>                | CSES       | ★★★★★  | So sánh Binary Lifting $O(Q \log N)$ vs Euler Tour RMQ $O(Q)$      |
| 15  | <b>CF 1446D2</b> | <a href="#">Frequency Problem (Hard)</a>          | Codeforces | ★★★★★  | Kỹ thuật chia căn tần số cân bằng độ phức tạp $O(N\sqrt{N})$       |
| 16  | <b>CF 438D</b>   | <a href="#">The Child and Sequence</a>            | Codeforces | ★★★★★  | Phân tích khấu hao phép Modulo: Giảm $\geq 50\%$ giá trị mỗi bước  |

## 💡 Phân Tích & Hướng Dẫn Chi Tiết (3-Tier Hints)

### Bài 1: CSES 1621 — Distinct Numbers

- **Đề bài:** Cho  $N$  số nguyên ( $N \leq 2 \times 10^5, x_i \leq 10^9$ ). Đếm số lượng phần tử phân biệt.
- **Gợi ý 1 (Nhận xét bản chất):**
- Có 3 cách tiếp cận với các mức độ phức tạp khác nhau:
  1. `std::set<int>`: Thời gian  $O(N \log N)$ , nhưng mỗi phần tử tốn 32...48 bytes bộ nhớ cho các con trỏ cây đỏ-đen, hệ số thời gian lớn do cache misses.
  2. `std::unordered_set<int>`: Trung bình  $O(N)$ , nhưng dễ bị hack  $O(N^2)$  nếu không dùng custom hash.
  3. `std::sort` + `std::unique`: Thời gian  $O(N \log N)$ , bộ nhớ  $O(1)$  phụ trợ, cache-locality hoàn hảo, chạy nhanh nhất trong thực tế.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Dùng `std::sort(a.begin(), a.end())`.
- Đáp án: `std::unique(a.begin(), a.end()) - a.begin()`.
- **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
- Thời gian:  $O(N \log N)$ , Bộ nhớ:  $O(N)$ .

### Bài 2: CSES 1095 — Exponentiation

- **Đề bài:** Cho  $a, b$ . Tính  $a^b \pmod{10^9 + 7}$  với  $b \leq 10^9$ .

- **Gợi ý 1 (Nhận xét bản chất):**
  - Thuật toán nhân liên tiếp  $O(b)$  tốn  $10^9$  phép tính  $\implies$  TLE chắc chắn.
  - Phân tích  $b$  thành tổng các lũy thừa của 2 (Lũy thừa nhị phân / Binary Exponentiation).
  - **Gợi ý 2 (Thuật toán & Tối ưu):**
  - Mỗi bước bình phương cơ số:  $a^{2k} = (a^k)^2$ ,  $a^{2k+1} = a \cdot (a^k)^2$ .
  - Số bước thực hiện chính xác bằng số bit của  $b$ :  $\lfloor \log_2 b \rfloor + 1 \leq 30$  phép nhân.
  - **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
  - Thời gian:  $O(\log b)$ , Bộ nhớ:  $O(1)$ .
  - Cạm bẫy: Ép kiểu `long long` khi nhân  $a \cdot a$  để tránh tràn số 32-bit trước khi modulo.
- 

### Bài 3: CSES 1083 — Missing Number

- **Đề bài:** Cho  $N - 1$  số nguyên phân biệt trong khoảng  $1 \dots N$ . Tìm số còn thiếu ( $N \leq 2 \times 10^5$ ).
  - **Gợi ý 1 (Nhận xét bản chất):**
  - Cách 1 (Mảng đánh dấu): Thời gian  $O(N)$ , Bộ nhớ  $O(N)$ .
  - Cách 2 (Tổng đại số):  $\frac{N(N+1)}{2} - \sum A_i$ . Thời gian  $O(N)$ , Bộ nhớ  $O(1)$ , nhưng  $N(N+1)$  có thể tràn kiểu `int`.
  - Cách 3 (Phép toán Bit XOR): Tính  $(1 \oplus 2 \oplus \dots \oplus N) \oplus (A_1 \oplus \dots \oplus A_{N-1})$ . Số nào xuất hiện 2 lần sẽ tự triệt tiêu về 0, số còn thiếu xuất hiện đúng 1 lần.
  - **Gợi ý 2 (Thuật toán & Tối ưu):**
  - Không cần lưu mảng, đọc trực tiếp từng số và XOR dồn vào biến `ans`.
  - **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
  - Thời gian:  $O(N)$ , Bộ nhớ:  $O(1)$  tuyệt đối!
- 

### Bài 4: CSES 1068 — Weird Algorithm

- **Đề bài:** Cho số nguyên  $n$ . Nếu  $n$  chẵn,  $n = n/2$ . Nếu  $n$  lẻ,  $n = 3n + 1$ . Lặp lại cho tới khi  $n = 1$ . In ra toàn bộ dãy số.
  - **Gợi ý 1 (Nhận xét bản chất):**
  - Đây là bài toán Giả thuyết Collatz ( $3n + 1$  Problem).
  - Về mặt toán học, độ phức tạp thời gian của thuật toán này vẫn là một bài toán mở chưa có lời giải chứng minh cận trên chặt chẽ!
  - **Gợi ý 2 (Thuật toán & Tối ưu):**
  - Với  $n \leq 10^6$ , dãy Collatz luôn dừng lại sau tối đa vài trăm bước.
  - **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
  - Cạm bẫy: Phép tính  $3n + 1$  có thể vượt quá giới hạn của `int` 32-bit trong quá trình lặp  $\implies$  Bắt buộc dùng `long long`.
- 

### Bài 5: CSES 1624 — Chessboard and Queens

- **Đề bài:** Đặt 8 quân hậu lên bàn cờ  $8 \times 8$  có một số ô bị cấm sao cho không có 2 quân hậu nào tấn công nhau.

- **Gợi ý 1 (Nhận xét bản chất):**
  - Đặt mỗi quân hậu trên một hàng độc lập từ  $1 \dots 8$ .
  - Không gian tìm kiếm là tập các hoán vị của 8 cột:  $8! = 40320$  trạng thái.
  - **Gợi ý 2 (Thuật toán & Tối ưu):**
  - Với 40320 trạng thái, thuật toán quay lui kèm nhánh cận kiểm tra đường chéo sẽ thực thi chỉ trong khoảng 0.001 giây!
  - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
  - Thời gian:  $O(N!)$ , Bộ nhớ:  $O(N)$ .
- 

## Bài 6: CSES 1640 — Sum of Two Values

- **Đề bài:** Cho mảng  $A$  gồm  $N$  số ( $N \leq 2 \times 10^5$ ). Tìm hai vị trí có tổng bằng  $X$ .
  - **Gợi ý 1 (Nhận xét bản chất):**
  - Phương pháp 1: Lưu cặp  $(A_i, i)$ , sắp xếp mảng theo giá trị  $O(N \log N)$ , sau đó dùng Hai con trỏ (Two Pointers) quét từ hai đầu trong  $O(N)$ .
  - Phương pháp 2: Dùng `std::unordered_map` lưu vị trí của  $X - A_i$ .
  - **Gợi ý 2 (Thuật toán & Tối ưu):**
  - Phương pháp Hai con trỏ có thời gian thực tế ổn định hơn và không tiêu tốn bộ nhớ động.
  - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
  - Thời gian:  $O(N \log N)$ , Bộ nhớ:  $O(N)$ .
- 

## Bài 7: CSES 1644 — Maximum Subarray Sum II

- **Đề bài:** Cho mảng  $A$ . Tìm tổng lớn nhất của đoạn con có độ dài nằm trong khoảng  $[a, b]$  ( $1 \leq a \leq b \leq N \leq 2 \times 10^5$ ).
  - **Gợi ý 1 (Nhận xét bản chất):**
  - Tổng đoạn  $A[i \dots j]$  bằng  $P_j - P_{i-1}$  với  $P$  là mảng tiền tố.
  - Với một cận phải  $j$  cố định, ta cần tìm  $\min P_{i-1}$  với  $j - b \leq i - 1 \leq j - a$ .
  - Đây là bài toán tìm Min trên cửa sổ trượt có kích thước  $b - a + 1$ .
  - **Gợi ý 2 (Thuật toán & Tối ưu):**
  - Sử dụng hàng đợi hai đầu đơn điệu (Monotonic Deque): Duy trì các giá trị  $P_k$  tăng dần.
  - Mỗi chỉ số được đẩy vào deque đúng 1 lần và lấy ra tối đa 1 lần  $\implies$  Chi phí khấu hao  $O(1)$  mỗi bước.
  - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
  - Thời gian:  $O(N)$ , Bộ nhớ:  $O(N)$ .
- 

## Bài 8: Codeforces 1213G — Path Queries

- **Đề bài:** Cho cây  $N$  đỉnh có trọng số cạnh. Có  $Q$  truy vấn  $w$ : Đếm số cặp  $(u, v)$  có trọng số cạnh lớn nhất trên đường đi  $\leq w$ .
- **Gợi ý 1 (Nhận xét bản chất):**
- Sắp xếp  $N - 1$  cạnh và  $Q$  truy vấn theo trọng số tăng dần.
- Áp dụng cấu trúc Disjoint Set Union (DSU) gộp các đỉnh khi trọng số cạnh  $\leq w$ .

- **Gợi ý 2 (Thuật toán & Tối ưu):**
- DSU có nén đường đi (Path Compression) và gộp theo kích thước (Union by Size) đạt độ phức tạp tiệm cận  $O(\alpha(N))$  mỗi thao tác (trong đó  $\alpha$  là hàm nghịch đảo Ackermann,  $\alpha(N) \leq 4$  với mọi  $N \leq 10^{80}$ ).
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian:  $O((N + Q) \log(N + Q))$ , Bộ nhớ:  $O(N + Q)$ .

## Bài 9: CSES 1081 — Maximum GCD

- **Đề bài:** Cho  $N$  số ( $N \leq 2 \times 10^5$ ,  $A_i \leq 10^6$ ). Tìm gcd lớn nhất của hai phần tử bất kỳ.
- **Gợi ý 1 (Nhận xét bản chất):**
- Duyệt ngược giá trị  $g$  từ  $10^6$  về 1.
- Đếm số lượng phần tử trong mảng là bội của  $g$ : Duyệt  $g, 2g, 3g, \dots \leq 10^6$ .
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Tổng số bước duyệt của vòng lặp là chuỗi điều hòa:

$$\sum_{g=1}^V \frac{V}{g} = V \ln V \approx 10^6 \times 13.8 \approx 1.4 \times 10^7 \text{ phép tính}$$

- Chạy trong chưa đầy 0.05 giây!
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian:  $O(V \ln V)$ , Bộ nhớ:  $O(V)$ .

## Bài 10: Codeforces 1338B — Edge Weight Assignment

- **Đề bài:** Gán trọng số dương cho các cạnh của cây sao cho XOR các cạnh trên đường đi giữa hai lá bất kỳ luôn bằng 0. Tìm số lượng trọng số phân biệt nhỏ nhất và lớn nhất có thể dùng.
- **Gợi ý 1 (Nhận xét bản chất):**
- XOR giữa hai lá bằng 0 khi và chỉ khi đường đi giữa hai lá có độ dài chẵn (nếu mọi cạnh cùng trọng số).
- Nếu tồn tại hai lá có khoảng cách lẻ, ta cần tối thiểu 3 trọng số. Nếu mọi cặp lá đều có khoảng cách chẵn, số trọng số tối thiểu là 1.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Chọn một đỉnh lá làm gốc, chạy một lượt DFS  $O(V + E)$  tính độ sâu chẵn/lẻ của các lá khác.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian:  $O(V)$ , Bộ nhớ:  $O(V)$ .

## Bài 11: CSES 1628 — Meet in the Middle

- **Đề bài:** Cho mảng  $N$  số ( $N \leq 40$ ,  $A_i \leq 10^9$ ) và số  $X$ . Đếm số tập con có tổng bằng  $X$ .
- **Gợi ý 1 (Nhận xét bản chất):**
- Thuật toán duyệt tập con ngây thơ:  $O(2^{40}) \approx 1.1 \times 10^{12}$  thao tác  $\implies$  TLE hoàn toàn.
- Chia mảng thành 2 nửa: Nửa trái  $N_1 = 20$  và nửa phải  $N_2 = 20$ .
- **Gợi ý 2 (Thuật toán & Tối ưu):**

- Sinh toàn bộ  $2^{20} \approx 10^6$  tổng của nửa trái, sắp xếp lại.
- Sinh từng tổng của nửa phải  $S_R$ , dùng tìm kiếm nhị phân ( `std::equal_range` ) đếm số lượng tổng nửa trái bằng  $X - S_R$ .
- Số phép tính giảm xuống:  $2 \times 2^{20} \approx 2 \times 10^6$  thao tác  $\implies$  Chạy trong 0.15 giây!
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian:  $O(2^{N/2} \cdot \frac{N}{2})$ , Bộ nhớ:  $O(2^{N/2})$ .

## Bài 12: Codeforces 1405E — Fixed Point Removal

- **Đề bài:** Cho mảng  $A$ . Phần tử  $A[i] = i$  là điểm bất động. Cho  $Q$  truy vấn bỏ  $x$  phần tử đầu và  $y$  phần tử cuối, tính số điểm bất động tối đa xóa được.
- **Gợi ý 1 (Nhận xét bản chất):**
- Thuật toán mô phỏng mỗi truy vấn tốn  $O(N) \implies$  tổng  $O(Q \cdot N) \approx 10^{10} \implies$  TLE.
- Quét Offline theo cận phải  $R$  kết hợp Cây Phân Đoạn (Segment Tree).
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Thay vì chặt nhị phân ngoài  $O(\log^2 N)$ , dùng Walk on Segment Tree tìm vị trí xóa trong  $O(\log N)$ .
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: Giảm từ  $O(Q \cdot N)$  xuống  $O((N + Q) \log N)$ .

## Bài 13: Codeforces 1144F — Graph Without Long Directed Paths

- **Đề bài:** Cho đồ thị vô hướng  $N$  đỉnh,  $M$  cạnh. Hãy định hướng các cạnh sao cho không có đường đi có hướng nào có độ dài  $\geq 2$ .
- **Gợi ý 1 (Nhận xét bản chất):**
- Đường đi có độ dài  $\geq 2$  xuất hiện khi có dạng  $u \rightarrow v \rightarrow w$ , tức đỉnh  $v$  vừa có cạnh đi vào, vừa có cạnh đi ra.
- Để không có đường đi độ dài 2: Mọi đỉnh chỉ có thể là **đỉnh nguồn (chỉ có cạnh đi ra)** hoặc **đỉnh đích (chỉ có cạnh đi vào)**.
- Điều này tương đương với: Đồ thị phải là **Đồ thị hai phía (Bipartite Graph)**!
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Chạy BFS/DFS tô 2 màu cho đồ thị trong  $O(V + E)$ . Nếu không tô được  $\implies$  xuất "NO".
- Nếu tô được: Cạnh nối giữa màu 0 và màu 1 luôn được định hướng từ 0  $\rightarrow$  1.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian:  $O(V + E)$ , Bộ nhớ:  $O(V + E)$ .

## Bài 14: CSES 1688 — Company Queries II

- **Đề bài:** Cho cây  $N$  đỉnh. Trả lời  $Q$  truy vấn tìm LCA của hai đỉnh  $u$  và  $v$  ( $N, Q \leq 2 \times 10^5$ ).
- **Gợi ý 1 (Nhận xét bản chất):**
- So sánh hai phương pháp kinh điển:
  1. **Binary Lifting:** Tiền xử lý  $O(N \log N)$ , mỗi truy vấn  $O(\log N) \implies$  Tổng thời gian  $O(N \log N + Q \log N)$ .

2. **Euler Tour + Sparse Table:** Traversal cây  $2N - 1$ , tiền xử lý  $O(N \log N)$ , mỗi truy vấn RMQ  $O(1) \implies$  Tổng thời gian  $O(N \log N + Q)$ .

• **Gợi ý 2 (Thuật toán & Tối ưu):**

• Khi  $Q = 10^6$ , phương pháp Euler Tour RMQ nhanh gấp **5-10 lần** so với Binary Lifting nhờ truy vấn  $O(1)$ .

• **Gợi ý 3 (Độ phức tạp & Cận bậc):**

• Thời gian:  $O(N \log N + Q)$ , Bộ nhớ:  $O(N \log N)$ .

---

## Bài 15: Codeforces 1446D2 — Frequency Problem (Hard)

• **Đề bài:** Cho mảng  $A$  gồm  $N$  phần tử ( $N \leq 2 \times 10^5$ ). Tìm độ dài đoạn con dài nhất có ít nhất 2 phần tử đạt tần số lớn nhất trong đoạn con đó.

• **Gợi ý 1 (Nhận xét bản chất):**

• Đặt  $M$  là phần tử có tần số lớn nhất toàn mảng.

• Phân chia đối tượng dựa trên ngưỡng tần số  $B = \lceil \sqrt{N} \rceil \approx 450$ :

◦ Các giá trị có tần số  $> B$ : Có tối đa  $N/B \leq \sqrt{N}$  giá trị khác nhau. Với mỗi giá trị, quy về bài toán tìm đoạn con có tổng bằng 0 trong  $O(N) \implies$  Tổng thời gian  $O(N\sqrt{N})$ .

◦ Các giá trị có tần số  $\leq B$ : Duyệt từng tần số  $f \in [1, B]$ , dùng Two Pointers tìm đoạn con dài nhất có tần số cực đại đúng bằng  $f$  trong  $O(N) \implies$  Tổng thời gian  $O(N\sqrt{N})$ .

• **Gợi ý 2 (Thuật toán & Tối ưu):**

• Cân bằng độ phức tạp của hai trường hợp giúp đạt cận trên hoàn hảo  $O(N\sqrt{N})$ .

• **Gợi ý 3 (Độ phức tạp & Cận bậc):**

• Thời gian:  $O(N\sqrt{N})$ , Bộ nhớ:  $O(N)$ .

---

## Bài 16: Codeforces 438D — The Child and Sequence

• **Đề bài:** Cho mảng  $A$ . Có 3 truy vấn: (1) Tính tổng đoạn, (2) Lấy modulo đoạn  $A[i] = A[i] \bmod x$ , (3) Cập nhật điểm  $A[k] = x$ .

• **Gợi ý 1 (Nhận xét bản chất):**

• Phân tích khấu hao phép modulo:

◦ Nếu  $A[i] < x \implies A[i] \bmod x = A[i]$  (không thay đổi).

◦ Nếu  $A[i] \geq x \implies A[i] \bmod x < A[i]/2$  (giá trị giảm ít nhất một nửa).

• **Gợi ý 2 (Thuật toán & Tối ưu):**

• Dùng Segment Tree lưu `max_val` và `sum`.

• Chỉ đệ quy vào các cây con có `max_val`  $\geq x$ .

• Mỗi phần tử chỉ có thể bị modulo thành công tối đa  $\log_2(\max A)$  lần trước khi về 0.

• **Gợi ý 3 (Độ phức tạp & Cận bậc):**

• Thời gian khấu hao toàn bài:  $O((N + Q) \log N \log(\max A))$ .

---



## **Competitive Programming Handbook**

*SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)*

*Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool*

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.